

IQNECT

by IQNOX for PTC Codebeamer

AI-Powered ALM Assistant
for Requirements Engineering

RDSG Gateway

INCOSE - REQ ENG Framework Team

System-Theoretic AI Governance
Architecture · STPA/STAMP Hardened

vs

Compare & Contrast

Requirements Intelligence · AI Governance · Enterprise Architecture

The Core Question: Two Very Different Problems

Both IQNECT and the RDSG Gateway engage requirements and AI — but they solve fundamentally different problems at different levels of the enterprise stack.

IQNECT asks:

How do we help requirements engineers write better requirements faster — with AI assistance inside the tools they already use?

Focus: Authoring productivity · Content quality · Traceability visualization

RDSG asks:

How do we make an AI-embedded enterprise system controllable — so requirements govern AI behavior rather than just describing system intent?

Focus: AI safety · Constraint enforcement · STPA/STAMP governance at scale

IQNECT: AI-Powered ALM Assistant for PTC Codebeamer

What It Is

IQNECT is a SaaS AI integration built by IQNOX for PTC's Codebeamer ALM platform. It adds an AI assistant layer directly into the requirements authoring environment — helping engineering teams in regulated industries write, analyze, decompose, and trace requirements more effectively.

Semantic Search

Find requirements by meaning, not keywords

Requirement Decomposition

Break high-level needs into downstream items

Content Analysis

AI reviews quality, completeness, and consistency

Trace Explorer

Interactive traceability diagram — upstream & downstream

Auto-Complete / Expand

AI suggests content aligned to project standards

Batch Analysis

Analyze entire trackers at once, not item-by-item

Architectural Context

Platform:	SaaS plugin on top of PTC Codebeamer
Deployment:	Cloud-hosted; connects to Codebeamer instances
Primary Users:	Systems engineers, requirements authors, QA teams
Industries:	Automotive, aerospace, medical device, defence
AI Role:	Assistant — AI helps humans do requirements work
Version:	v1.4 (August 2025)
Standards Config:	Company vocabulary & industry guidelines configurable
Tailored Config:	Business rules and org-specific context injected

RDSG Gateway: System-Theoretic AI Governance Architecture

What It Is

The RDSG is an enterprise AI governance architecture that treats requirements as first-class ontology individuals — not documents — that actively govern AI agent behavior through SHACL/SWRL constraint enforcement, STPA/STAMP safety analysis, and a formal hierarchical control structure.

Requirements Ontology

SHACL-validated, SWRL-enforced — live constraint engine

UCA / STPA Integration

4 UCA types formally constrain all AI control actions

JTBD / Persona Anchoring

DesiredOutcome metrics become AI success criteria

Simplex Action Gateway

Human handshake required for any Irreversible Zone 6 act

5-Stage Pipeline

Stage 0→Step 5 compilation: SPO → Compiled AI Directive

AcceptanceCriteria Gates

10-criterion Service Facade validation before delivery

Architectural Context

Platform:	Architecture framework — deployable across enterprise
Foundation:	OWL ontology · SHACL shapes · SWRL rules · GraphDB
Primary Users:	Enterprise architects, AI governance, compliance, ops
Industries:	Any regulated enterprise — pharma, finance, defence, mfg
AI Role:	Controlled process governed by requirement constraints
Version:	v2.0 STPA/STAMP Hardened Edition
Safety Theory:	STAMP · STPA · Unsafe Control Action analysis
Standards:	ISO 31000 · NIST AI RMF · EU AI Act · DO-178C ready

Requirements Philosophy: Document vs. Ontology Individual

IQNECT: Requirement as Document

- › Requirements live in Codebeamer as work items — text artifacts with fields, attributes, and version history
- › AI assists in authoring: auto-completing text, checking quality, suggesting decompositions
- › Traceability is visualized (Trace Explorer) but managed manually by engineers
- › Standards alignment configured by loading vocabulary and guidelines into the AI's context
- › Requirements drive human decisions — the system helps humans work on requirements
- › Semantic search finds requirements by meaning — but requirements don't execute constraints

RDSG: Requirement as Ontology Individual

- › Requirements are typed OWL individuals — forPersona, describesJob, hasCriterion, enforcedBy — living in the triplestore
- › Requirements govern AI behavior in real time: SHACL shapes and SWRL rules derived from active requirements constrain every AI response
- › Traceability is end-to-end: from UCA analysis → UCAConstraint → SHACL shape → SWRL rule → CAD element → validated response
- › STPA integration: UCAs are formalized per requirement. Safe-state actions are machine-executable policy, not documentation
- › Requirements are the control structure — they are not inputs to AI, they are the enforcement envelope AI operates inside
- › AcceptanceCriteria are executable tests evaluated by the Service Facade before any output is delivered

The AI Role: Assistant vs. Controlled Process

IQNECT

AI helps humans do requirements work better

The human is in the loop. AI is the assistant.

RDSG

Requirements govern what the AI is permitted to do

AI is the controlled process. Requirements are the controller.

What the AI receives

A Codebeamer context and the engineer's prompt

A Compiled AI Directive — typed governance object with UCA profile, AcceptanceCriteria, RDQE, and Zone 6 flags

Who validates output

The requirements engineer reviews AI suggestions

The Service Facade evaluates all responses against 10 AcceptanceCriteria before delivery — not the human

What 'good' means

Content quality metrics: completeness, consistency, grammar, decomposition coverage

DesiredOutcome metrics defined in the ontology: formally tested by the Service Facade against the RCB

Failure response

Engineer reviews and corrects AI suggestions manually

Response recycled with structured correction instructions; after 3 Must-priority failures, RequirementsReview is triggered automatically

Traceability: Within ALM vs. End-to-End Governed Chain

IQNECT Traceability

Scope:	Within Codebeamer — upstream and downstream item relationships in the ALM project data model
Mechanism:	Trace Explorer: interactive diagram expanding relationships. Suspect traces highlighted for review.
Depth:	Requirement → Test case → Change request → Implementation item
Human role:	Engineer navigates and resolves traces; AI assists in identifying gaps and suspects
Standard support:	Configurable per company vocabulary — supports ASPICE, ISO 26262, DO-178C contexts
Limitation:	Does not trace from requirements to runtime AI behavior — traceability stops at the system boundary

RDSG End-to-End Traceability

Scope:	From stakeholder need → ontology individual → SHACL shape → SWRL rule → CAD element → agent behavior → validated output
Mechanism:	Stage 0 RCB build creates the traceability chain; every CAD element is annotated with the Requirement ID that mandated it
Depth:	UCA analysis → UCAConstraint → ControlRequirement → enforcedBy SHACL → Service Facade AcceptanceCriterion → delivery
Human role:	Domain stewards define requirements; governance layer and Service Facade enforce — human is not in the runtime validation loop
Standard support:	ISO 31000 · NIST AI RMF · EU AI Act Art. 9 · SOX · GDPR — enforced as ControlRequirements with SHACL shapes
Capability:	Every delivered response carries a validation record traceable to requirements in the triplestore — queryable on demand

Safety & Governance: Content Quality vs. STPA/STAMP Controllability

IQNECT ← Content quality & authoring productivity System-theoretic AI safety & controllability → RDSG		
Governance Model	Human-in-the-loop. AI suggestions are reviewed and approved by requirements engineers. Governance = human judgment applied to AI output.	Requirements-as-controller. SHACL shapes and SWRL rules enforce constraints automatically. Governance = structural enforcement before output reaches the human.
Safety Theory	None explicitly. Safety comes from regulated-industry pedigree of Codebeamer and configured company standards — not from formal safety analysis of the AI itself.	STPA/STAMP. Prof. Leveson's System-Theoretic Process Analysis identifies Unsafe Control Actions across 4 failure modes. UCAConstraints become machine-executable ControlRequirements.
Irreversible Actions	Not in scope. IQNECT operates within the requirements authoring environment — it does not trigger external writes or irreversible enterprise actions.	Simplex Action Gateway. Zone 6 actions classified as Irreversible are suspended pending a 5-step human handshake with identity verification, attestation, and immutable audit logging.
Failure Handling	Suggestion quality varies. Engineers may accept poor suggestions without formal checking. No structured escalation path when AI-generated requirements prove inadequate at test.	10-criterion AcceptanceCriteria pipeline at the Service Facade. Non-passing responses recycled with structured correction objects. After 3 Must-priority failures, automated RequirementsReview triggered.

Scope & Scale: ALM Plugin vs. Enterprise Architecture

Enterprise AI Governance Landscape

RDSG Gateway

Enterprise AI governance architecture

Zone 1: Prompt Engineering

Zone 2: Semantic Gateway (5-stage pipeline)

Zone 3: Ontology & Rules Store

Zone 4: Service Facade

Zone 5: AI Agents + Vector DBs

Zone 6: Legacy / External Systems (Simplex Gate)

RDSG spans all enterprise AI zones — from persona intent to external system writes — across all domains, all personas, and all AI agents.

IQNECT

ALM plugin for Codebeamer

Codebeamer Platform

Requirements Items / Trackers

AI Semantic Engine

Trace Explorer

Quality Analysis

IQNECT operates at the ALM tool layer — a SaaS plugin for one platform, serving one phase of the engineering lifecycle (requirements authoring).

Head-to-Head: 12-Dimension Comparison

Dimension	IQNECT	RDSG Gateway
Problem Solved	AI-assisted requirements authoring in ALM	Requirements-governed AI behavior across the enterprise
Architectural Type	SaaS plugin on top of PTC Codebeamer	Enterprise governance architecture — platform-agnostic
Requirements Role	Documents to be authored better with AI help	Ontology individuals that govern AI behavior as live constraints
AI Role	Assistant to human requirements engineers	Controlled process governed by SHACL/SWRL requirement constraints
Traceability Scope	Within Codebeamer ALM data model	From UCA analysis → constraint → AI directive → validated output
Safety Theory	None explicit — relies on human review	STPA/STAMP: 4 UCA types formally defined per FunctionalJob
Irreversibility Control	Not in scope — no external writes	Simplex Action Gateway: 5-step handshake with immutable audit log
Validation Method	AI suggests; engineer reviews and accepts	Service Facade: 10-criterion AcceptanceCriteria before delivery
Standards Integration	Configurable vocabulary and guidelines	Executable SHACL shapes enforcing ISO 31000, NIST AI RMF, EU AI Act
Deployment Model	Cloud SaaS — connects to Codebeamer instance	Architecture framework — deploys across enterprise stack
Primary Value	Faster, higher-quality requirements authoring	Controllable AI systems with formal safety guarantees at scale
Maturity	Commercial product — v1.4 (August 2025)	Architecture framework — DRAFT v2.0 STPA/STAMP Edition

Where They Could Converge: Complementary Strengths

IQNECT and RDSG operate at different phases of the same value chain. They are not competitors — they are potentially sequential partners. IQNECT improves the quality of requirements that the RDSG then governs.



IQNECT zone

Better Requirements In → Better Governance Out

IQNECT's AI-assisted authoring produces requirements that are more complete, consistently structured, and better decomposed. When these requirements are formalized as RDSG ontology individuals, the SHACL `sh:RequirementCompleteness` shape is more likely to pass on first validation — reducing governance gaps and `GOVERNANCE_GAP` flags in CADs.

RDSG UCA Elicitation → IQNECT Codebeamer Items

The RDSG's Stage 1 UCA elicitation block (4 UCA types per FunctionalJob) could seed Codebeamer tracker items via IQNECT's Codebeamer integration. Safety requirements derived from STPA analysis would automatically appear as traceable items in the ALM environment, creating a formal bridge between hazard analysis and requirements management.

IQNECT Trace Explorer → RDSG Traceability Matrix

IQNECT's Trace Explorer visualizes ALM-internal relationships. The RDSG's SPARQL-generated Requirements Traceability Matrix extends that view from the ALM boundary through the gateway pipeline to validated AI outputs. Together they provide a complete picture: from stakeholder intent authored in Codebeamer to governed AI delivery.

Limitations & Gaps: What Each Does Not Address

IQNECT: What It Doesn't Address

Does not govern AI behavior at runtime — only assists in authoring phase

No formal safety analysis of the AI layer itself (no STPA/UCA framework)

Requirements remain documents — not executable constraints on AI agents

No Irreversible action protection — no concept of Zone 6 or Simplex Gateway

Validation is human-judgment dependent — no automated AcceptanceCriteria

Limited to Codebeamer ecosystem — not portable across enterprise AI landscape

Traceability ends at the ALM boundary — does not trace AI response quality

No formal process model synchronization — cannot detect human-AI misalignment

RDSG: What It Doesn't Address

Complex to implement — requires ontology engineering, SHACL/SWRL expertise, and triplestore infrastructure

Not a plug-and-play product — is an architecture framework that must be built and operated

Does not assist in requirements authoring — assumes requirements are well-defined before entering the pipeline

Learning curve is steep — STPA/STAMP analysis, OWL ontology design, and SPARQL competence required

No ALM integration out of the box — requirements must be formalized into the ontology separately

UI/UX for elicitation interface must be built — the architecture specifies it but does not provide it

Vendor neutral by design — no pre-built connector to Codebeamer, Jira, DOORS, etc.

Complexity scales with domain coverage — each new domain requires ontology, shapes, rules, and UCA analysis

Strategic Positioning

Two different answers to two different questions — at two different levels of the enterprise AI stack

If you need to write better requirements faster

IQNECT delivers immediate value for engineering teams already using Codebeamer. It reduces authoring time, improves quality, and surfaces traceability gaps — all within a familiar tool. The ROI is operational productivity for requirements engineers.

If you need to govern AI behavior at enterprise scale

The RDSG Gateway is the right architecture. It does not compete with ALM tools — it governs what happens after requirements leave those tools and become the basis for AI-driven operations. The ROI is trust, auditability, and defensible AI governance under regulatory scrutiny.

If you need both — and regulated enterprises do

The two architectures are complementary. IQNECT improves requirements quality in Codebeamer. The RDSG formalizes those requirements as governed constraints on AI behavior. Together they close the loop from human intent to governed AI output with full traceability at every step.